Removal Stability: Eviction, Tombstone Bounds and Heartbeats for the Data Topics of Rings

Ryan J. Kung

Abstract

A Rings data topic is a replicated last-writer-wins element set whose removes are collected at a clock horizon (#872, #913). Four open defects share one cause: nothing states which actions may drop a remove before it is stable, nor how many removes a carrier may hold. Storage eviction drops removes and resurrects removed elements (#920); a sync peer can grow a carrier's removes past one transport message (#921); the fetch cache was specified without a linearization (#922); and registry heartbeats emit one remove each, about 25 KB per registrant (#918). We define stability, state one survival law for removes, and prove no resurrection under it. We factor a carrier into an element side and a remove side, give each its point of the durable-records authority lattice, and so confine eviction to elements. We bound the remove side by a count enforced by refusal, prove the carrier fits one message, and show that a forged sync delta simulates a writer. We give the cache's linearization, and replace the heartbeat by the re-append of a signed lease, which removes nothing and reads nothing.

I. Problem

#920
: The byte budget of the data store evicts the least recently written carrier, whatever it holds. An evicted remove lets a stale replica's sync re-add the element it covered.

#921
: A data carrier's removes have no count or byte bound, and a sync delta may carry any number of them. The assertion that a full carrier fits one transport message counts elements only.

#922
: A cache read that retires an entry can overwrite or delete a value that a concurrent cache put just stored.

#918
: A registry descriptor's bytes change on every heartbeat, so every heartbeat removes the previous descriptor. Removes are retained for $H + \sigma$, which leaves about 200 removes per registrant, and every registrant reads all of them on every heartbeat.

#920 and #921 concern the survival and the size of a remove. #918 is the dominant producer of removes. #922 is the one store that holds carriers and is not replicated. Section II fixes the model, Section III the survival law, and Sections IV–VII one law per issue. Section VIII orders the implementation.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created October 24, 2026. This document specifies when a removal of a data-topic element of Rings is stable, what may drop it, how large it may grow, how the fetch cache is linearized, and what a registry heartbeat writes, before 1.0 (rings-core, modules dht::entry, dht::chord::storage and storage; rings-node, module registration). It is the law for issues #918, #920, #921 and #922 of RingsNetwork/rings, and extends the durable-records law [1].

II. Model

Definition 1 (Carrier). A data carrier is $x = (V, D, \rho, \beta)$: adds V : Bytes $\rightharpoonup$ Dot, removes D : Digest $\rightharpoonup$ Dot, a register $\rho \in \mathrm{Ver}_\perp$ (the overwrite floor) and a bound $\beta \in \mathbb{N}_\perp$. A dot leads with its hybrid time $\tau(d)$. The remove $(e, r) \in D$ covers the add $(v, d) \in V$ iff $h(v) = e \wedge d \leq r$, with h Keccak-256. x is normalized iff no add is covered or below ρ and no remove is below ρ. The join $\sqcup$ is the pointwise max followed by normalization (crdt.rs). It is a semilattice on normalized carriers [2].

Definition 2 (Horizon, retirement, liveness). With H the element horizon ($6 \cdot 10^6$ ms) and σ the skew tolerance (3000 ms), as in #913:

$$\begin{aligned} \mathrm{retire}_t(x) = x &\setminus \{(v, d) : t \geq \tau(d) + H\} \\ &\setminus \{(e, r) : t \geq \tau(r) + H + \sigma\} \\ &\setminus \{\rho : t \geq \tau(\rho) + H + \sigma\}, \\ s(x) &= \max\{\tau(r) : r \in \operatorname{ran} D \cup \{\rho\}\} + H + \sigma, \\ \mathrm{live}_t(x) &\iff \beta \neq \perp \wedge (t < \beta \vee t < s(x)). \end{aligned}$$

retire_t is a join homomorphism and $\mathrm{retire}_t \circ \mathrm{retire}_s = \mathrm{retire}_{\max(s,t)}$.

Assumption 1 (Clocks). Any two node clocks differ by at most σ. Admission at clock c refuses a delta that carries a version with $\tau > c + \sigma$.

Actors. A writer is any node: a data topic has no write authority, so any node may Extend, Tombstone or Overwrite any topic. A syncer is any node that sends a delta (a hand-off, a repair, a lookup reply), which the receiver admits by the admission law alone. An owner holds a replica of a placement in its data store. The fetch

cache is a mirror: it is not a replica, it never acknowledges, and it spreads only by read-repair.

Program actions and loss events. A program action is a transition of the store that the program performs: a join, an operation, a retirement, an ack-gated deletion, eviction, a cap, or a refusal. A loss event is what the platform or the operator does (Definition 4 of [1]). The distinction matters because a writer controls program actions through the writes it sends, but it cannot cause a loss event.

III. Stability

Definition 3 (Stable). A remove (e, r) is stable at clock t iff $t \geq \tau(r) + H + \sigma$, and likewise the register ρ iff $t \geq \tau(\rho) + H + \sigma$. A carrier's remove side is stable at t iff $t \geq s(x)$.

Lemma 1 (A stable remove covers only retired adds). If (e, r) is stable at the clock t_p of some node, then every add (v, d) it covers is retired at the clock t_q of every node at the same instant.

Proof. $t_q \geq t_p - \sigma \geq \tau(r) + H \geq \tau(d) + H$, since $d \leq r \Rightarrow \tau(d) \leq \tau(r)$. The register case is the same, with the adds below ρ. □

Definition 4 (Holder). Owner p holds (e, r) at time t iff its replica has $D(e) \geq r$. The mirror is never a holder.

Law 1 (Remove survival). An unstable remove or register leaves a holder's replica only by (i) retire_t at its stability, (ii) an ack-gated move whose receiver holds it, or (iii) a loss event. No other program action drops it. In particular, no eviction, cap or truncation does.

Theorem 1 (No resurrection). Under Law 1, let p hold (e, r) at t_0, and let no loss event strike p before $\tau(r)+H+\sigma$. Then no read at p after t_0 serves an add covered by (e, r).

Proof. Before stability, $D_p(e) \geq r$ by Law 1, because no admissible transition lowers it: a join takes the pointwise max, and a move keeps it at the receiver. Normalization then drops every covered add from each value p stores or reads. From stability on, Lemma 1 retires every covered add, and every read applies retire_t. □

Lemma 2 (Replication answers loss events, not eviction). A remove held by k holders leaves the system before it is stable only if all k drop it. Under Law 1, that takes a loss event at each of them. If eviction may drop removes, one writer drops it from all k with no loss event.

Proof. A writer may write to every placement of any topic, so it can fill the byte budget of each of the k owners. Eviction is a function of the writes received. □

So #920 breaks Law 1 through a program action that a writer can drive, and it breaks it at every replica at once. The remaining sections discharge the law and bound what it retains.

IV. Eviction and the factored carrier

Definition 5 (Sides). $\mathsf{E}(x) = (V, \emptyset, \bot, \beta)$ and $\mathsf{R}(x) = (\emptyset, D, \rho, 0)$.

The remove side carries the least stamped bound, 0. So a remove side read alone is live only through $s(x)$, and it answers a lookup as a miss (bound_live_at is false). It never shadows a replica that holds elements.

Lemma 3 (Factorization). For normalized x, $x = \mathsf{E}(x) \sqcup \mathsf{R}(x)$. For every E' whose adds are a subset of those of some carrier the replica held and every $R' \sqsupseteq \mathsf{R}(x)$, $E' \sqcup R'$ contains no add that x's removes or register drop.

Proof. No add of x is covered or below ρ, so normalization of the union drops nothing, and $\max(\beta, 0) = \beta$. For the second claim, normalization of $E' \sqcup R'$ drops every add that R' covers or floors, and R' covers or floors at least what $\mathsf{R}(x)$ does. □

Law 2 (Eviction acts on elements). Eviction maps $x \mapsto \mathsf{R}(x)$. It may drop the element side of any carrier, and the whole carrier iff $\mathsf{R}(\text{retire}_t(x)) = \bot$.

Proposition 1 (What eviction may drop). Law 2 satisfies Law 1, and it is the largest eviction that does: dropping any unstable remove or register violates it.

Proof. $x \mapsto \mathsf{R}(x)$ keeps D and ρ. A remove that retire_t already dropped is stable. Dropping an unstable (e, r) is a program action outside (i)–(iii). □

Law 3 (Authority per side). A data carrier is stored as two records under one key. E goes to the data store at $(\mathsf{Cch}, \mathsf{Reb})$, which is unchanged (Table 1 of [1]). R goes to a removal store at $(\mathsf{Dur}, \mathsf{Ret})$, RecordAuthority::Retained.

Justification. An absent remove side means "never removed here, or retired at stability". That is the meaning Ret, whose obligations are N (no eviction) and D. N is Law 2. D follows from Admissibility (Law 1 of [1]). D also turns (ii) into a move under crashes: the receiver acknowledges a hand-off only after its flush. The element side keeps Reb, because a lost add is an availability loss that replication repairs. □

Law 4 (Write order). $\text{get}(k) = \mathsf{E}_k \sqcup \mathsf{R}_k$. $\text{put}(k, x)$ first writes $\mathsf{R}(x)$, or removes the record if $\mathsf{R}(x) = \bot$, and does so only if it differs from the stored R_k. It then writes $\mathsf{E}(x)$. It returns Ok only after both, and an $\mathsf{Err}(\mathsf{Exhausted})$ from the removal store leaves both records unchanged. $\text{rm}(k)$ removes E_k and then R_k.

Proposition 2 (Every crash state is safe). After any crash, $\text{get}(k)$ serves no add that a remove or register the replica made durable covers or floors, until that remove or register is stable.

Proof. D gives $\mathsf{R}_k \in \{\mathsf{R}(x_m), \mathsf{R}(x_{m+1})\}$ for the last two removal-store writes. Under Cch, E_k is the element side of some earlier write, or $\bot$. A remove side changes only by joins, which grow it, and by retire_t, which drops stable parts; so the durable R_k covers or floors every unstable

remove of every earlier carrier. The claim then follows from Lemma 3, and Lemma 1 covers the stable parts. A crash between the two removals of rm leaves a remove side alone, which is live until stable and answers no lookup. □

Cost. One flush per write that changes the remove side. An Extend never changes it, and after Section VII neither does a heartbeat.

Budget. The removal store refuses and never evicts (Law 8 of [1]). Its budget B_{R} is shared by every topic at the owner, so it is partitioned by origin (Law 6 of Section V). An operation refused for room returns Exhausted to its writer. A refused sync join is never acknowledged, so the sender keeps its copy and retries.

Browsers. A browser's crash class is claimed, Dur°, and a store that holds state for others is admissible at $(\mathsf{Dur}^{\circ}, \mathsf{Ret})$ when $r \geq 2$ holders keep it and every repair period restores r copies (Law 12 of [1], as amended by #1056). A remove side is held by every owner of its topic, $r =$ `DATA_REDUNDANT` $= 6$ by default, and repair restores the replica set. So a browser opens the removal store in IndexedDB at $(\mathsf{Dur}^{\circ}, \mathsf{Ret})$, with strict commit and refusal. By Proposition 9 of [1], a remove is lost only if every holder is lost within one repair period. A deployment configured with $r = 1$ makes the browser removal store an inadmissible point, as Law 12 makes any single-holder browser custody; native owners at $(\mathsf{Dur}, \mathsf{Ret})$ need no redundancy.

IndexedDB that fails to open. Only then does a browser keep the remove side in memory, refusing at its budget: $(\mathsf{Vol}, \mathsf{Ret})$, logged at warn. Each restart is the total loss of Proposition 2 of [1]: the node comes back as a fresh replica. Theorem 1 then fails at that node alone, and by Lemma 2 the remove survives at every other holder.

Mirror. The fetch cache is $(\mathsf{Vol}, \mathsf{Reb})$. Its count bound evicts whole carriers, which ends a residency (Section VI). It drops no holder's remove, and its read-repair delivers to a holder only values that the holder's removes still cover.

V. The tombstone bound

Definition 6 (Cap). $T_{\max} = D_{\max} =$ `ENTRY_DATA_MAX_LEN` $= 1024$ removes per data carrier.

Law 5 (Remove admission). (a) A peer-supplied delta δ with $|D_\delta| > T_{\max}$ is not admissible. Like the clock bound, this is a predicate on the delta. (b) A join or an operation whose result $y = \mathrm{retire}_t(x \sqcup \delta)$ has $|D_y| > T_{\max}$ is refused with $\mathsf{Err}(\mathsf{Exhausted})$, and the slot is unchanged. A data carrier's removes are never truncated.

Proposition 3 (Refuse, not truncate). Truncating a data carrier's removes violates Law 1. A relay inbox may truncate (#913), because it has one holder and one ack-gated relocation at a time.

Proof. Truncation drops an unstable remove by a program action, and Lemma 2 applies, since a writer controls which removes exceed the cap. A relay inbox has no second replica that could keep a stale add. □

Proposition 4 (One message). Let w_T, w_D and w_ρ be the postcard suprema of one remove, one dot and one register (142, 110 and 106 B: a dot is a varint $u128$, two 42-character DIDs and a varint $u32$; a digest is 32 B). Let $\lambda = 3$ be the length prefix of a 32 KiB element and w_0 the carrier's fixed fields. Every admitted data carrier encodes in at most

$$W = w_0 + D_{\max}(P_{\max} + \lambda + w_D) + T_{\max} w_T + w_\rho \approx 33.82\,\mathrm{MB},$$

with $W_{\mathsf{R}} = w_0 + T_{\max} w_T + w_\rho$ for its remove side. The carrier law is $W +$ `MAX_PAYLOAD_ENVELOPE_OVERHEAD` $\leq$ `TRANSPORT_MAX_SIZE` (60 MB), asserted at compile time. Each w is a constant witnessed with equality by a test, as the envelope reserves are.

Proof. The cap $D_{\max}$, the per-element bound $P_{\max}$ and Law 5 bound the counts, and each field is at most its supremum. □

The margin is wide: removes cost 145 KB at the cap, against 26 MB of headroom. We pick $T_{\max} = D_{\max}$ for the size of a lookup reply, not for the message bound.

Honest rate. The cap admits $T_{\max}/(H+\sigma) \approx 10$ removes per minute per topic, sustained. A topic that removes faster, such as a chat log that deletes each message, is refused past the cap: its writer gets Exhausted, and nothing is silently dropped. Raising $T_{\max}$ would cost every lookup reply of a busy topic w_T per remove, and every owner that much of its removal budget. An application that needs a higher rate shards the topic: k topics t_i, with each element written to $t_{h(v) \bmod k}$, admit k times the rate, and since their keys differ they place at different owners.

Lemma 4 (A sync delta simulates a writer). Let δ be admissible at clock c. Some writer, using at most $2|D_\delta| + |V_\delta| + 1$ operations stamped at clocks $\leq c$, produces at an owner a carrier that has the same coverage and remove count as $x \sqcup \delta$, up to dots in $(c, c+\sigma]$.

Proof. Write each add with Extend and the register with Overwrite. Take a remove (e, r). If $e = h(v)$ for an element v that some carrier holds, then Tombstone(v) covers what (e, r) covers at dots up to c. Otherwise no held add has digest e (collision resistance), so (e, r) covers nothing, and the pair Extend(v_e), Tombstone(v_e) for a fresh v_e adds one remove that covers nothing held. Admission bounds every $\tau(r) \leq c + \sigma$. □

Proposition 5 (Forgery through sync adds no power). A syncer cannot exceed W (Law 5). It covers an add only as a writer's Tombstone would, with a dot up to σ later. It can fill one carrier's removes, which refuses further removes on that carrier at that owner until $c+\sigma+H+\sigma$. By Lemma 4, any writer can do the same with $O(T_{\max})$ operations.

So the defect of #921 closes for one carrier. A remote peer can still exhaust the remove capacity of a topic, but every writer could already do that, since a data topic has no write authority and its element cap already lets a writer displace the topic's elements.

Isolation across topics. The removal store is one budget for every topic at an owner, and it refuses instead of evicting, so junk held there stays until it is stable. By Lemma 4, one writer can place Extend(v_e), Tombstone(v_e) pairs on fresh topics at an owner and, without a further bound, refuse every honest remove on every topic there for $H + \sigma$. A per-topic share does not prevent this: the junk topics are fresh topics, each under its own cap. The share must follow the sender.

Definition 7 (Charge). Owner p charges each unstable remove (t, e) it holds to an origin $o_p(t, e)$: the authenticated sender of the transition that last raised $D_t(e)$ at p. That is the operation's writer, as the shell verified its signature, or the signer of the sync message. A sync from a node that p's own view names as an owner of t's placement (the predecessor handing off its range, or a replica peer) is charged to the holder class $\star$ instead. o_p is local to p: it is stored beside each remove in the removal store, never sent, and recounted at open. Let $c_p(o) = |\{(t, e) : o_p(t, e) = o\}|$ over unstable removes.

Law 6 (Origin share). A transition from sender $o \neq \star$ that would raise $c_p(o)$ above $S = T_{\max}$ is refused with Err(Exhausted), and the slot is unchanged. The store refuses any transition past its total $B_{\mathsf{R}} = 64\,T_{\max}$ removes ($\approx 9.3\,\mathrm{MB}$ at w_T). A transition that raises no $D_t(e)$ is never charged, and a charge is released when its remove is retired at stability.

Proposition 6 (Isolation). Let an honest writer w remove an element of topic t at owner p. The remove is refused only if (i) t holds $T_{\max}$ unstable removes, (ii) w's own share is exhausted, $c_p(w) = S$, or (iii) the total is reached, which requires removes charged to at least $\lceil (B_{\mathsf{R}} - c_p(\star))/S \rceil - 1$ origins other than w, or removes charged to $\star$.

Proof. Laws 5 and 6 refuse for exactly three reasons: the topic cap, the sender's share, and the total. The total is $c_p(\star) + \sum_{o \neq \star} c_p(o)$ with each $c_p(o) \leq S$, so reaching it with $c_p(w) < S$ needs the stated number of other origins. An origin is the authenticated sender, so no peer can charge w: a forged sync is charged to its own signer. □

Case (iii) needs either 63 distinct identities or a node that p's view makes an owner of the topic. The first is a Sybil attack and the second is a Byzantine storage owner, and SECURITY.md lists both as non-goals. Capacity exhaustion by one writer is thus confined to its own share and to topics it fills to $T_{\max}$; this is the availability residual stated in SECURITY.md. An honest writer may hold S unstable removes per owner, about 10 per minute per owner, sustained.

Proposition 7 (Refusal is safe). Suppose owners p and q refuse each other's deltas because $|D_p \cup D_q| > T_{\max}$. Then neither violates Theorem 1. They converge once the union fits, which happens by $\max \tau(r) + H + \sigma$ over the removes present if no new remove arrives.

Proof. A refusal changes no slot, so no holder drops a remove. An add that p serves and q has removed is a remove p never held. That is a stale replica, not a resurrection. □

VI. The fetch cache

Definition 8 (Cache machine). The cache maps $c : K \rightharpoonup X$. Its transitions run under one lock (the cache transition), each at its own clock t:

$$\begin{aligned} \mathrm{put}(v):&\ \ c[k] \leftarrow \mathrm{retire}_t\big(\mathrm{live}_t(c[k]) \sqcup v\big),\\ \mathrm{obs}(o):&\ \ c[k] \leftarrow \big(\mathrm{live}_t(c[k]) \text{ or } \bot\big).o,\\ \mathrm{evict}(k'):&\ \ c[k'] \leftarrow \bot, \end{aligned}$$

the last inside a put of a new key at the count bound. A read at t returns $\mathrm{retire}_t(c[k])$ when it is live and its bound has not elapsed, and it changes nothing.

Law 7 (Pure reads). A cache read is a function of one state and the reader's clock, and it never writes or removes. Every state change is one transition under the cache transition. The store orders single gets and puts atomically.

Proposition 8 (Linearizable; no lost put). Each transition linearizes [3] at its store put, and each read at its store get. After put(v) at k, $c[k] \sqsupseteq \mathrm{retire}_t(v)$ holds until the next transition at k. That transition is a put or obs, which keeps $c[k]$ ascending in $\sqsubseteq$ up to retirement (#982), or the eviction of k, which ends the residency.

Proof. Transitions are totally ordered by the lock, and reads have no write effect. So the history is the lock-order sequence of transitions, with every read placed at its get. The trace of #922 needs a read that stores, and no read stores. The clocks need not be monotone in lock order: by composition a state stored at an earlier clock is a less retired version of the one at the later clock, and every read projects at its own clock. □

This is the state since #913 made the cache read return its projection only. A dead cached value stays until a put replaces it or the count bound evicts it. It costs one of `LOCAL_CACHE_CAPACITY` slots and is served as absent. Read-your-writes (#982) holds per residency. #922 needs a test that witnesses Proposition 8, not a code change.

VII. Heartbeats

Today a descriptor signs *heartbeat_at* and *expires_at*, so its bytes change at every heartbeat. Each heartbeat therefore reads the registry, appends the new descriptor and removes the old one. With interval $I = 30\,\mathrm{s}$ that leaves $(H+\sigma)/I \approx 200$ removes, about 25 KB, per registrant, and N registrants read about $N^2 \cdot 25\,\mathrm{KB}$ per interval.

The issue's direction, stress-tested. #918 proposes to drop the timestamps and derive liveness from the element's dot. A dot is stamped by whoever writes, and it is unsigned: by Lemma 4 a syncer can forge one too. Any node that re-appends a dead registrant's observed bytes would then refresh its dot, and so keep it listed forever.

Freshness that a peer cannot forge needs a signed bound. The lease below keeps the bytes stable and keeps that bound signed.

Definition 9 (Lease descriptor). A lease descriptor d signs the registrant's identity fields, its process epoch, its capabilities and a lease end ℓ, and nothing that changes between heartbeats. Its bytes are fixed for the lease. The lease length is $\Lambda = L = \texttt{DEFAULT_TTL_MS}$ (10 min).

Law 8 (Heartbeat). At each heartbeat at clock t, a registrant writes exactly Extend($[d]$) for its current lease descriptor d. It writes no remove and reads nothing. If $t + 2I \geq \ell$, or if a signed field changed (restart, capabilities), it first signs d' with $\ell' = t + \Lambda$ and appends d' in place of d. It never removes d. A remove is written only on withdrawal (advertising turned off), at most once per lease.

Law 9 (Reader liveness). Let $\tau^*(d)$ be the dot of d in the fetched carrier. A reader at clock t lists, for each DID, the descriptor with the greatest ℓ among those that verify and satisfy $t < \ell$ and $t < \tau^*(d) + T + \sigma$, where T is the registry TTL (90 s).

Proposition 9 (Cost). Heartbeats write no removes. A registrant writes at most one remove per withdrawal within $H+\sigma$, so registries stay far below $T_{\max}$. A registrant leaves at most $\lceil H/(\Lambda - 2I) \rceil + 1 = 13$ descriptors within the horizon. The cap keeps the $D_{\max}$ newest dots, and a heartbeat refreshes the current descriptor's dot, so superseded leases are dropped first. A heartbeat performs no registry read, which removes the N^2 read traffic.

Proof. Law 8 writes Extend only, and an Extend of bytes already held raises their dot (pointwise max) without a remove. A new lease is signed at most once per $\Lambda - 2I$. □

Proposition 10 (Ghost bound). Let a registrant stop at t_s. If no node re-appends its bytes, it is listed until at most $t_s + T + \sigma$. Against any writer or syncer, it is listed until at most $\ell \leq t_s + \Lambda$.

Proof. Its last honest dot has $\tau^* \leq t_s$, which gives the first bound. A re-append or a forged dot can raise τ^*, but not the signed ℓ, which gives the second. □

The cost is the trade-off of this law. A replayed registrant can stay listed for up to $\Lambda = 10$ min, where today it stays at most T. In exchange, heartbeats produce no removes, and the per-heartbeat registry read and its N^2 traffic disappear. A larger Λ retains fewer descriptors and lengthens the ghost window. The bound applies to both registries (online nodes and onion exits).

VIII. Implementation order

PR	Issue	Law
1	#922	Law 7; a test witnessing Prop. 8
2	#918	Def. 9, Laws 8, 9 (rings-node)
3	#921	Def. 6, Law 5; the assertion of Prop. 4
4	#920	Laws 2–4, 6; the removal store

PR 2 precedes PR 3. Once PR 3 lands, today's heartbeats would hit $T_{\max}$ with about five registrants per topic, and their removes would be refused. PR 4 needs PR 3, because the removal store's budget is sized by W_{R}. PR 1 is independent.

Operator steps, in the CHANGELOG under Unreleased. PR 2 changes the descriptor encoding, so old descriptors no longer decode. Readers skip them, and they retire within H; this needs no step. PR 4 adds the removal store (native directory and browser database dht-removals beside the data store) and needs no step either. A carrier written before PR 4 keeps its removes in the data store until it is next written, and the read join (Law 4) still applies them. An operator who wants Law 2 to hold from the first eviction removes the data store while the node is stopped; replication rebuilds it.

IX. Summary of laws

Question	Answer
when stable	$t \geq \tau(r) + H + \sigma$ (Def. 3)
what may drop it	retire_t, a move, a loss event (Law 1)
what eviction drops	the element side (Law 2)
where removes live	removal store, $(\mathsf{Dur}, \mathsf{Ret})$ (Law 3)
how many	$\leq T_{\max}$, refused past it (Law 5)
store isolation	per-origin share (Prop. 6)
how large	$W \leq$ one message (Prop. 4)
forged by sync	simulates a writer (Lemma 4)
cache order	pure reads, locked transitions (Law 7)
heartbeat writes	Extend of a signed lease (Law 8)

References

[1] R. J. Kung, "Durable records: One presence and survival law for the local stores of a Rings node," papers/durable-records.tex, https://github.com/RingsNetwork/rings, 2026, issue #1024.

[2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems (SSS 2011), ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[3] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," ACM Transactions on Programming Languages and Systems, vol. 12, no. 3, pp. 463–492, 1990.